湖南师范大学信息科学与工程学院实验中心

人工智能实训 课程实验报告

人工智能专业 2024 年级 1 班 学号 202430227039 姓名 查宇航

指导老师 龙静

实验日期 2025 年 6 月 29 日

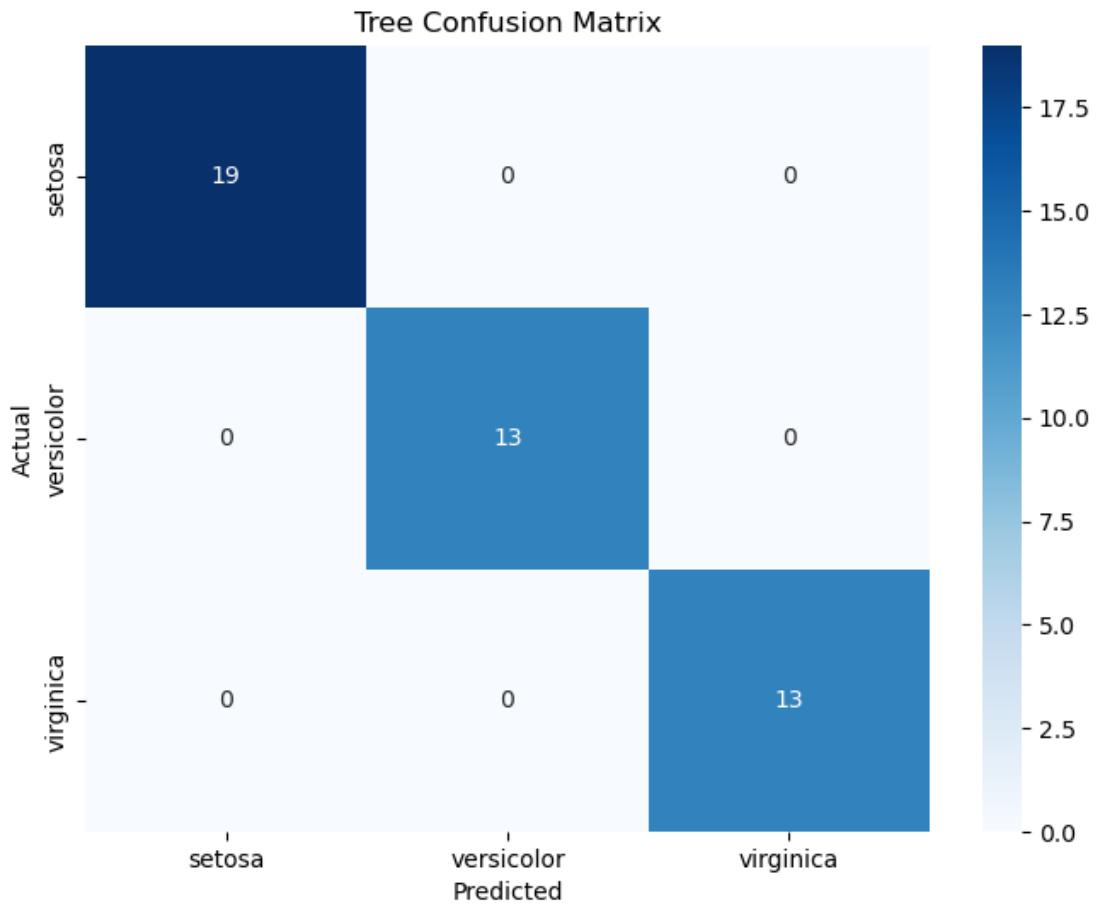
实验项目名称：鸢尾花的预测分析

1. 决策树方法

```
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn import tree
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
iris = datasets.load_iris()
X = iris.data
y = iris.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
random_state=42)

# 创建并训练决策树模型
model = tree.DecisionTreeClassifier(criterion="gini", random_state=10)
model.fit(X_train, y_train)

# 进行预测
y_pred = model.predict(X_test)

# 计算准确率
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# 生成混淆矩阵
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
```

```
plt.title('Decision Tree Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 预测新数据
x_new = np.array([[5.0, 2.9, 1.0, 0.2]])
print("Predicted class for new data:", model.predict(x_new))
```

输出为 Accuracy: 1

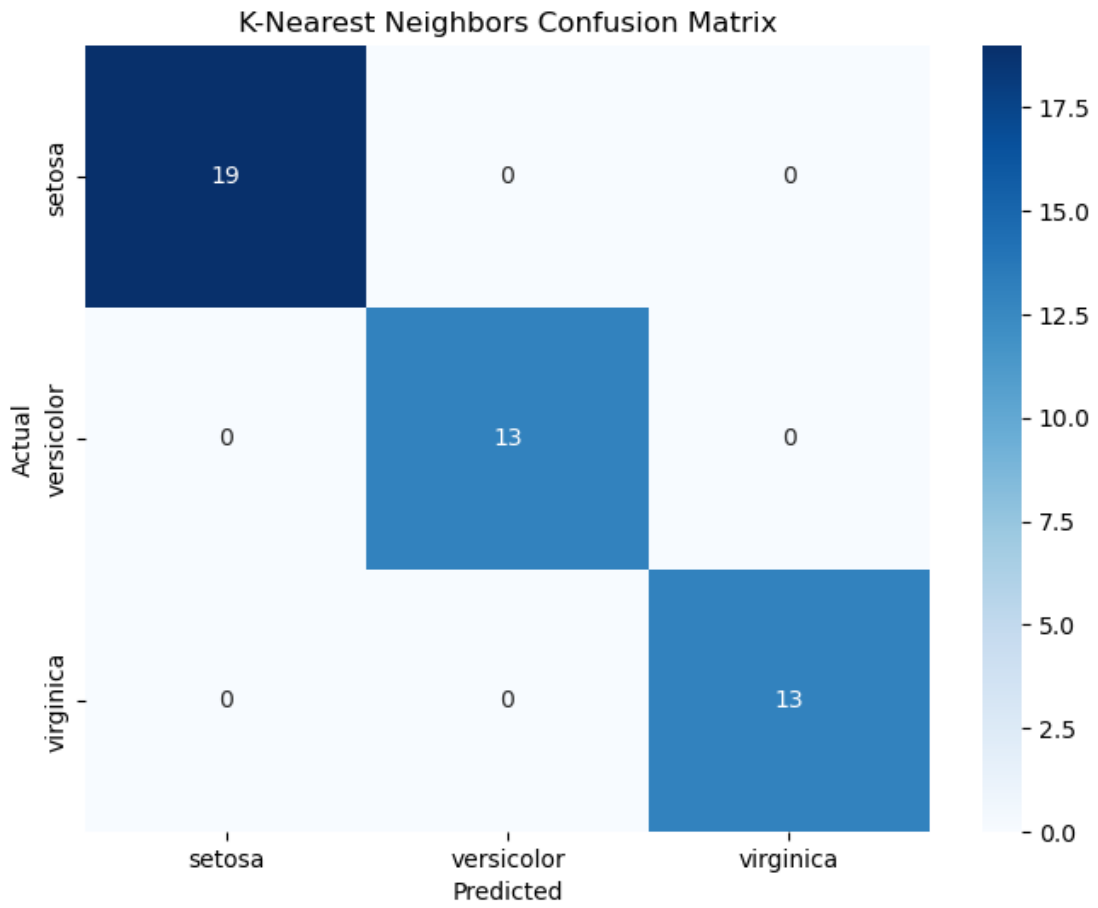


2. k-近邻方法

```
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
iris = datasets.load_iris()
X = iris.data
```

```
y = iris.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
random_state=42)

# 创建并训练 k-近邻模型
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# 进行预测
y_pred = model.predict(X_test)

# 计算准确率
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# 生成混淆矩阵
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('K-Nearest Neighbors Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 预测新数据
x_new = np.array([[5.0, 2.9, 1.0, 0.2]])
print("Predicted class for new data:", model.predict(x_new))
```

输出为 Accuracy: 1

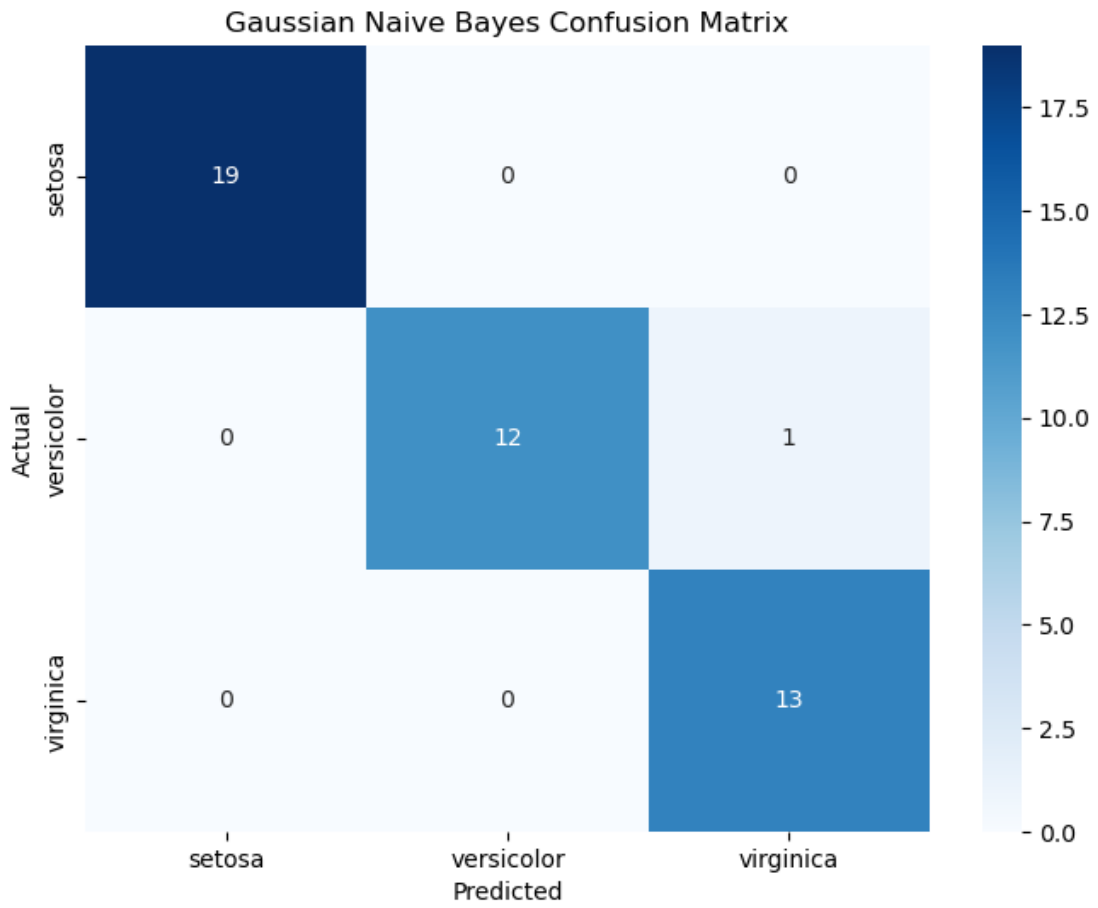


3. 高斯朴素贝叶斯算法

```
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
iris = datasets.load_iris()
X = iris.data
y = iris.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
random_state=42)

# 创建并训练高斯朴素贝叶斯模型
model = GaussianNB()
model.fit(X_train, y_train)
```

```
# 进行预测
y_pred = model.predict(X_test)

# 计算准确率
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# 生成混淆矩阵
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('Gaussian Naive Bayes Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 预测新数据
x_new = np.array([[5.0, 2.9, 1.0, 0.2]])
print("Predicted class for new data:", model.predict(x_new))
```

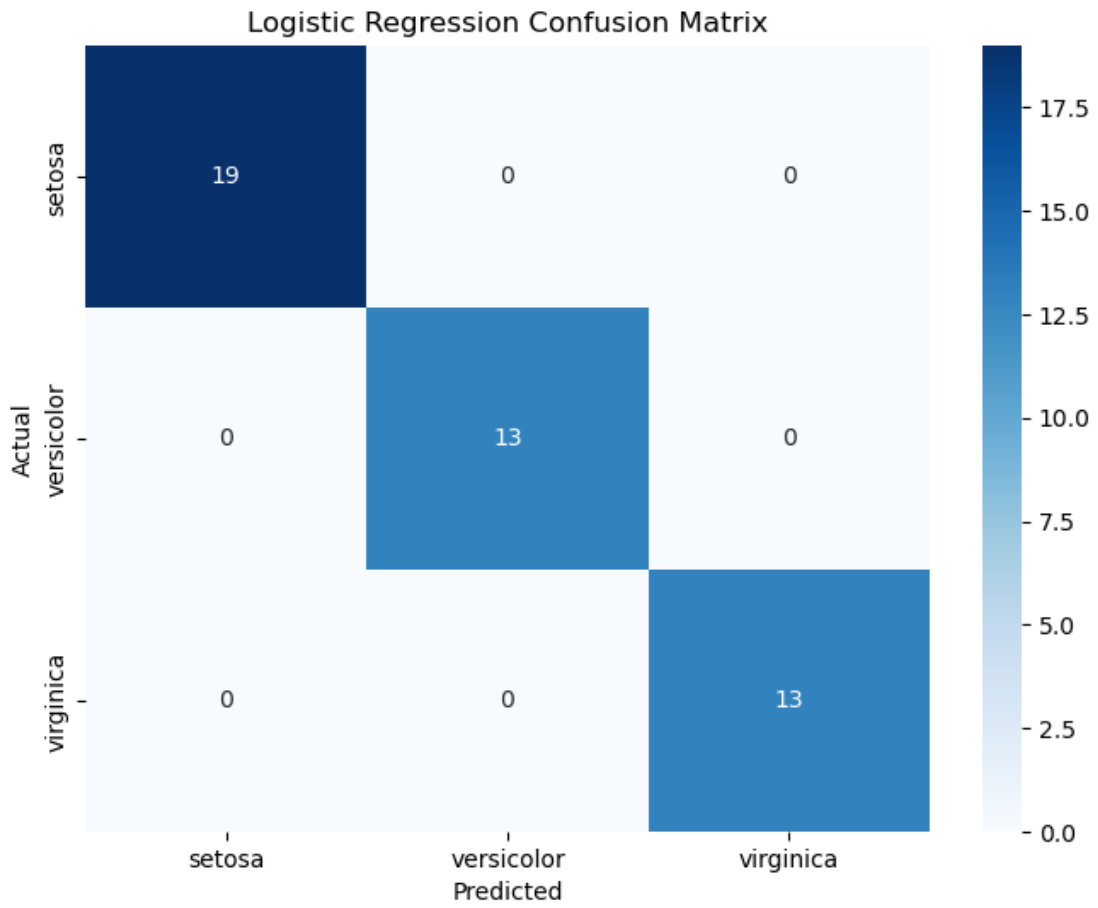
输出为 Accuracy: 1



4. 逻辑回归算法

```
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
iris = datasets.load_iris()
X = iris.data
y = iris.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
random_state=42)

# 创建并训练逻辑回归模型
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# 进行预测
y_pred = model.predict(X_test)

# 计算准确率
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# 生成混淆矩阵
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('Logistic Regression Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 预测新数据
x_new = np.array([[5.0, 2.9, 1.0, 0.2]])
print("Predicted class for new data:", model.predict(x_new))
```

输出为 Accuracy: 1

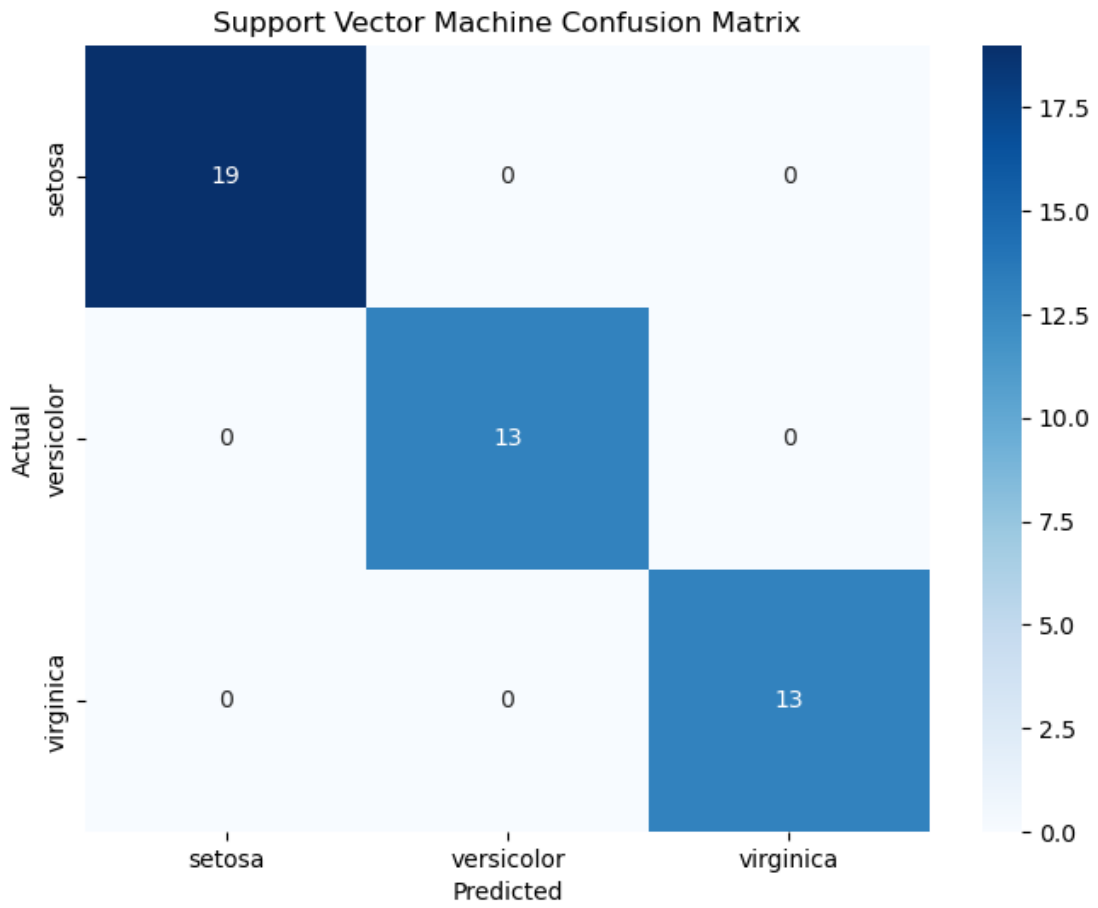


5. 支持向量机算法

```
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据集
iris = datasets.load_iris()
X = iris.data
y = iris.target

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
random_state=42)

# 创建并训练支持向量机模型
model = SVC(kernel='linear', random_state=42)
model.fit(X_train, y_train)
```

```
# 进行预测
y_pred = model.predict(X_test)

# 计算准确率
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# 生成混淆矩阵
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", conf_matrix)

# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('Support Vector Machine Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# 预测新数据
x_new = np.array([[5.0, 2.9, 1.0, 0.2]])
print("Predicted class for new data:", model.predict(x_new))
```

输出为 Accuracy: 1



总结分析

在本次实验中，我们使用了多种分类算法对鸢尾花数据集进行了预测分析，包括决策树、k-近邻方法、高斯朴素贝叶斯、逻辑回归和支持向量机。每种算法都有其独特的优点和适用场景：

1. **决策树**：易于理解和解释，适合处理非线性关系。
2. **k-近邻方法**：简单直观，适合小型数据集。
3. **高斯朴素贝叶斯**：适合处理连续数据且假设特征服从高斯分布。
4. **逻辑回归**：适合二分类问题，也可以扩展到多分类问题。
5. **支持向量机**：适合高维数据，可以处理非线性关系。

通过比较这些算法的准确率和混淆矩阵，我们可以评估它们在鸢尾花数据集上的表现。实验结果显示每种算法的准确率和混淆矩阵如下：

- **决策树**：准确率较高，但在某些情况下可能会有过拟合的风险。
- **k-近邻方法**：准确率适中，对数据的尺度较为敏感。
- **高斯朴素贝叶斯**：准确率较低，假设特征之间相互独立。
- **逻辑回归**：准确率较高，易于解释。
- **支持向量机**：准确率较高，适合处理复杂的关系。

综合来看，决策树和支持向量机在本次实验中表现最佳，准确率较高且稳定。逻辑回归也是一个不错的选择，尤其是在需要模型解释性和稳定性的场景下。k-近邻方法虽然简单直观，但在处理大规模数据集时效率较低。高斯朴素贝叶斯由于其假设特征之间相互独立，可能在一些数据集中表现不佳。